

Module 31: Exceptions in Class Design

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Module 15 taught throw, try and catch. This module teaches students to design classes that stay correct when exceptions pass through them. They build their own exception hierarchy rooted in `std::runtime_error`, including exceptions that carry data the catcher can use; trace exactly what is destroyed when a constructor throws (and why raw resources then leak); learn why destructors are `noexcept` and must never let an exception escape; and meet the basic, strong and no-throw guarantees with concrete techniques for the strong guarantee. RAII is revealed as the reason modern C++ is practical to make exception-safe, and rethrowing and translating exceptions complete the picture.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Design an exception hierarchy derived from `std::runtime_error`, with one root per system.
- Write exceptions that carry data, and catch them by const reference in the right order.
- Explain exactly what is and isn't destroyed when a constructor throws.
- Explain why destructors must never let an exception escape, and use the `close()-plus-destructor` pattern.
- Describe the no-throw, strong and basic guarantees, and implement the strong guarantee.
- Use RAII so that clean-up happens automatically during stack unwinding.
- Rethrow with `throw`; and translate low-level exceptions into domain exceptions.

Prerequisites

- Part 2 Module 15 (exception handling).
- Modules 23 and 26 (RAII and smart pointers).

Materials & Instructor Prep

- 31-exceptions-in-class-design/examples/ 01 to 06; 02_constructor_throws.cpp and 04_strong_guarantee.cpp are central.
- The table of exception-safety guarantees from the README on a slide.

Key Vocabulary

Term	Definition
Exception hierarchy	A set of exception classes derived from one root, so callers can catch at the level of detail they need.
Stack unwinding	Leaving each function and block between a throw and its catch, destroying their local objects on the way.
No-throw guarantee	The operation never throws (noexcept) - required of destructors, move operations and swap.
Strong guarantee	If an exception is thrown, the operation has no effect: the object is exactly as before ('commit or roll back').
Basic guarantee	If an exception is thrown, nothing leaks and the object is still valid, but its contents may have changed.
Rethrow (throw;)	Inside a catch block, throws the same exception object again, keeping its real type.
Exception translation	Catching a low-level exception and throwing a higher-level one with more useful context.

Lesson Outline & Instructor Talking Points

1. Your own exception hierarchy (20 min)

- Run examples/01_exception_hierarchy.cpp. Show `InsufficientFundsError`'s `shortfall()` being used by the catcher - data, not just a message.
- Discuss the handler order and why catching by value would slice the exception (Module 28).

2. When a constructor throws (15 min)

- Have students predict examples/02_constructor_throws.cpp's output for the invalid patient: is `~Patient` printed? Are the Parts destroyed?
- Show the Recorder example from the README: a raw `new[]` member followed by a throw leaks, because the destructor never runs. The fix is the Rule of Zero.

Instructor note: This is where Modules 23-26 pay off: students who follow the Rule of Zero get exception-safe constructors for free.

3. Destructors must not throw (10 min)

- Run examples/03_destructor_noexcept.cpp. Explain `std::terminate` when two exceptions are in flight, and that destructors are `noexcept` by default.
- Present the pattern: a `close()` that can throw for callers who care, and a destructor that swallows errors.

4. The exception-safety guarantees (25 min)

- Run examples/04_strong_guarantee.cpp. Read the three results aloud: broken, valid-but-changed, untouched.
- Teach the technique: do everything that might throw on the side, then commit with operations that can't throw (including swap).
- Run examples/05_raii_vs_leak.cpp to show RAII cleaning up during unwinding without any try/catch for clean-up.

Instructor note: Give students the vocabulary 'strong guarantee' and expect them to use it in project comments from now on.

5. Rethrow and translate (10 min)

- Run examples/06_rethrow_translate.cpp. Contrast throw; with throw e; (which slices).
- Show translation adding context at a layer boundary, and close with guidance on noexcept.

Common Student Misconceptions

Misconception	How to Address It
"If a constructor throws, its destructor cleans up."	The object never existed, so its destructor never runs. Only fully constructed members and bases are destroyed - which is why members should manage themselves.
"catch (std::exception& e) first is fine because it catches everything."	It also catches the specific types, so later, more specific handlers never run. Order handlers from most specific to most general.
"throw e; and throw; are the same."	throw e; throws a copy as the caught type, slicing derived exceptions. throw; rethrows the original object.
"Exception safety means wrapping everything in try/catch."	Most exception safety comes from RAII and ordering operations correctly; try/catch is for handling, not clean-up.

Exercises & Module Project

Exercise: 31-exceptions-in-class-design/exercises/exercise1.cpp - a school library with a LibraryError hierarchy (including a data-carrying LoanLimitError), a validating Member constructor, and borrowMany() with the strong guarantee. Project (three tracks): Generic - course enrolment; EE - load switching with an OvercurrentError carrying measured current; Biomedical - medication orders with DoseLimitError and AllergyConflictError. Each adds a hierarchy, a strong-guarantee batch operation, translated CSV loading with line numbers, a log-and-rethrow layer, a noexcept-safe destructor, and guarantee comments on every public method.

Facilitation guidance: Test each project's batch operation with a failure in the middle and confirm the state is unchanged afterwards. Check that one corrupted CSV line is reported with its line number and does not stop the rest. Read the guarantee comments for honesty - a correctly labelled basic guarantee is better than a wrongly claimed strong one.

Discussion & Comprehension Check Questions

1. Why should your exception classes derive from `std::runtime_error` (or `std::exception`)?

Guidance: So generic handlers catching `std::exception` still catch them, and `what()` works as expected.

2. How do you give a batch operation the strong guarantee?

Guidance: Check and prepare everything that might throw first, without changing the object, then commit using operations that cannot throw.

3. Why is RAII essential for exception safety?

Guidance: Stack unwinding destroys local objects, so RAII objects release their resources automatically on the exception path without extra code.

Assessment Ideas

- Students classify three given functions as no-throw, strong, basic or no guarantee, and fix the one with no guarantee.
- Students design a three-level exception hierarchy for a scenario and write the ordered catch blocks.

Answer Key Notes

See [31-exceptions-in-class-design/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Our classes are now robust. Next module brings templates back - combining them with classes, inheritance and C++20 concepts to build reusable, generic components."

Instructor Tips

- If time is short, protect constructor throws (step 2) and the strong guarantee (step 4).
- Ask students to say which guarantee each of their methods gives - the vocabulary sharpens their thinking.